.UBAfiuba
FACULTAD DE INGENIERÍA

Circuitos Lógicos Programables

Especialización en Sistemas Embebidos

Trabajo práctico final

Generador PWM configurable por UART

Implementación en VHDL sobre la placa Arty Z7-10 (Zynq-7000)

Autores: Espinola Maximiliano,
Sottile Victoria
Cohorte: 26Co2026
Año: 2026

1. Introducción y objetivo

El objetivo del trabajo es implementar en VHDL un bloque de hardware digital que genera una señal PWM cuya frecuencia y ciclo de trabajo (duty cycle) se configuran desde una computadora mediante comandos de texto enviados por UART. El sistema responde a cada comando con OK o ERR por la misma UART.

El diseño se implementó en la placa Arty Z7-10 (Zynq-7000), usando el reloj de 125 MHz del PL y una UART a 115200 baudios en formato 8N1. Se simularon los componentes y el bloque principal, se sintetizó e implementó el diseño y se verificó su funcionamiento en la placa con ayuda de un VIO y un ILA de Vivado.

2. Explicación de lo implementado

Desde una terminal serie se envían comandos ASCII terminados en CR o LF. El receptor UART reconstruye cada byte, el intérprete de comandos (parser) lo analiza y actualiza la configuración, y el generador PWM aplica la nueva frecuencia o duty cycle. En paralelo, el parser arma la respuesta y la envía a la PC a través del transmisor UART.

- **Camino de comando:** uart_rx → uart_cmd_parser → pwm_generator.
- **Camino de respuesta:** uart_cmd_parser → uart_tx → PC.

Datos del diseño

Dato	Valor
Reloj del sistema	125 MHz (período de 8 ns)
Baud rate / formato	115200 baudios, 8N1
Ciclos de reloj por bit	125 000 000 / 115 200 = 1085 (≈ 8,68 µs por bit)
Tiempo por byte	10 bits ≈ 86,8 µs
Respuesta OK / ERR	"OK\r\n" (4 bytes, ≈ 347 µs) / "ERR\r\n" (5 bytes, ≈ 434 µs)

Protocolo de comandos

Comando	Efecto
`F1` / `F2` / `F3`	Selecciona una de las tres frecuencias de PWM.
`D<NN>` (por ejemplo `D75`)	Fija el duty cycle en porcentaje (0 a 100, hasta 3 dígitos).
Respuesta válida	`OK\r\n`

Comando	Efecto
Respuesta inválida	`ERR\r\n` (la configuración anterior se mantiene)

Frecuencias de PWM

Comando	freq_sel	Período (ciclos de 8 ns)	Frecuencia PWM
`F1`	`"01"`	125 000	1 kHz
`F2`	`"10"`	12 500	10 kHz
`F3`	`"11"`	1 250	100 kHz

La frecuencia resulta de 125 MHz dividido el período,

$$Frecuencia\ PWM\ =\ \frac{125Mhz}{período}$$

y el umbral del duty cycle es

$$Umbral\ Duty\ Cicle\ =\frac{T.\ duty}{100}$$

El resultado brinda el número exacto de ciclos de reloj que la señal debe permanecer en estado alto (ON) dentro de cada período.

Estos períodos corresponden a la versión de pwm_generator.vhd corregida para cumplir timing (ver 6.4).

3. Diagramas en bloques

La Figura 1 muestra el sistema completo, con el VIO y el ILA usados para la prueba en placa. La Figura 2 detalla la máquina de estados del parser y la Figura 3 el generador PWM.

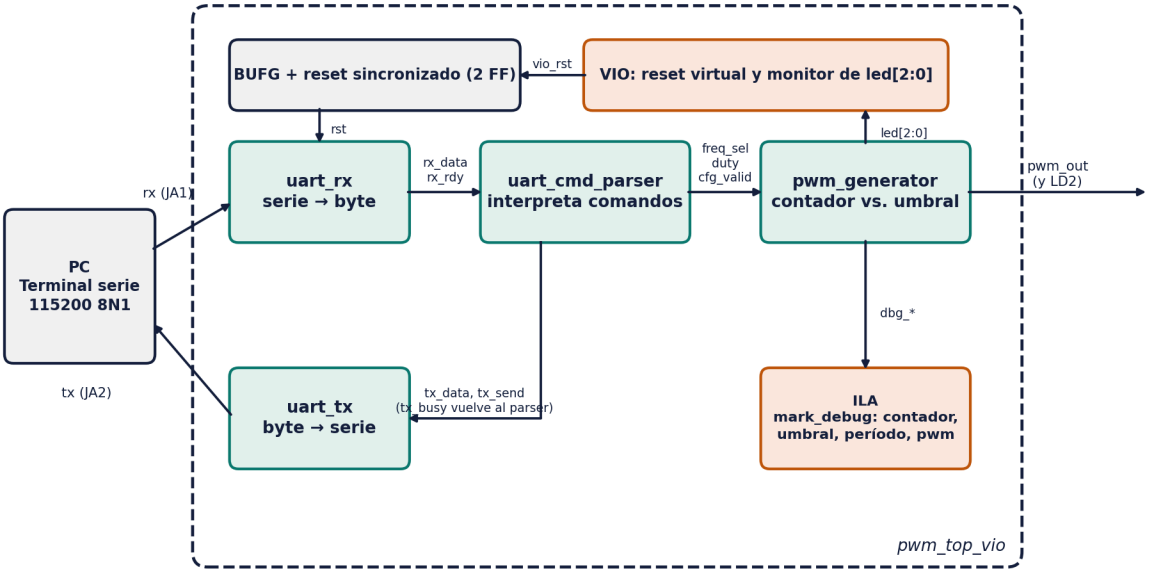


Figura 1. Diagrama en bloques del sistema (pwm_top_vio).

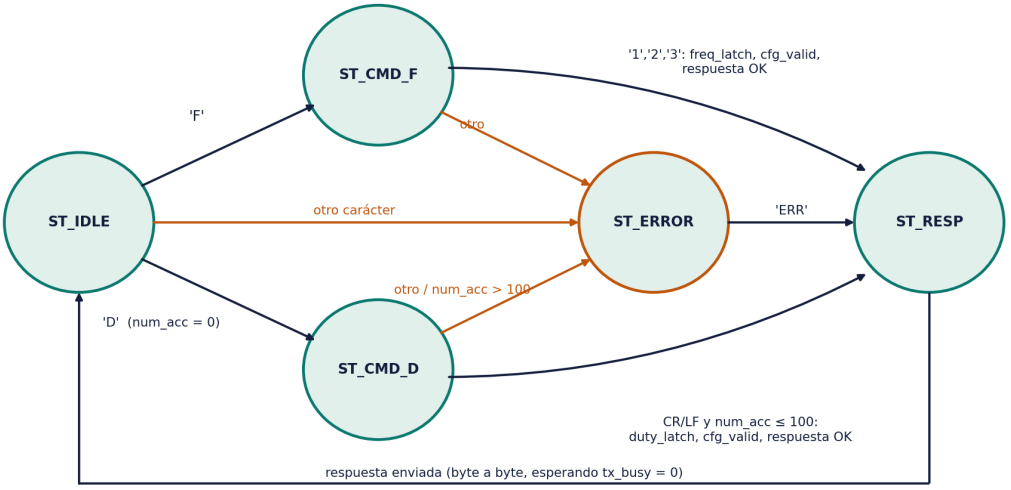


Figura 2. Máquina de estados del parser de comandos.

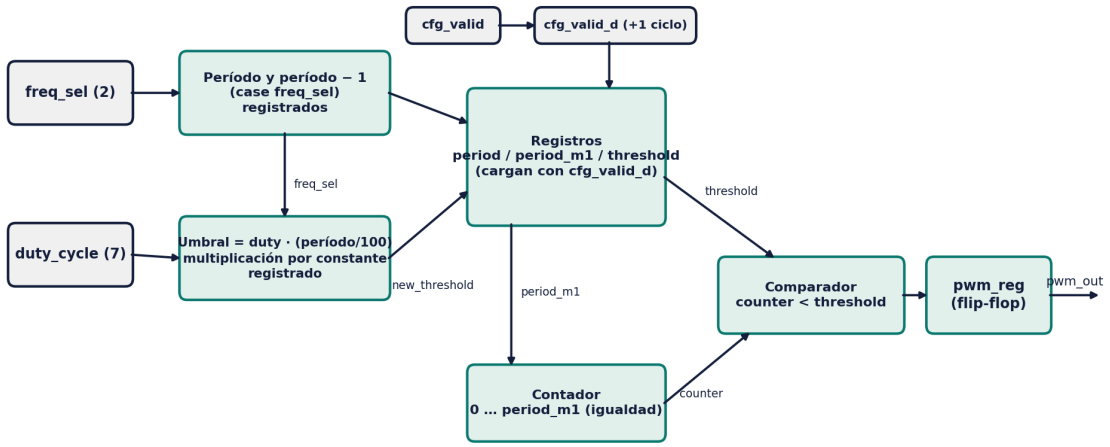


Figura 3. Diagrama en bloques del generador PWM (versión con cálculo registrado).

4. Descripción de los módulos

4.1 uart_rx

Convierte la línea serie en bytes paralelos y avisa con un pulso de un ciclo (data_rdy).

- Sincronizador de 2 flip-flops para la entrada asíncrona rx, que evita problemas de metaestabilidad.
- Máquina de estados IDLE, START, DATA y STOP. En START espera medio bit y confirma que la línea sigue en cero; si no, descarta el falso start.
- Muestrea cada bit en el centro (cada 1085 ciclos), con el bit menos significativo primero.
- Limitación: espera el tiempo del bit de stop pero no verifica que valga 1, por lo que no detecta errores de trama.

4.2 uart_tx

Serializa un byte cuando recibe un pulso de un ciclo en send y mantiene busy en 1 mientras transmite.

- Máquina de estados IDLE, START, DATA y STOP, de 1085 ciclos por bit; envía el bit menos significativo primero.
- busy es el handshake con el parser: no se envía otro byte hasta que vuelve a 0.

4.3 uart_cmd_parser

Es el intérprete de comandos. Reconoce F1 a F3 y Dxx, actualiza los latches de configuración y arma la respuesta.

- Acumula el número del duty en decimal: `num_acc = num_acc·10 + dígito`. Con D75: 7, luego 75; al llegar CR o LF se valida (≤ 100) y se aplica.
- `cfg_valid` es un pulso de un ciclo que indica al generador que cargue la nueva configuración.
- La respuesta se envía byte a byte en ST_RESP, solo cuando `tx_busy = 0` y `tx_send_reg = 0`; la segunda condición cubre el ciclo en que uart_tx todavía no levantó busy.
- Valores iniciales: frecuencia seleccionada con "01" y duty de 50 %.
- Limitación: un duty con más de 3 dígitos no está contemplado.

4.4 pwm_generator

Genera la salida comparando un contador con un umbral. Esta es la versión corregida para cumplir timing a 125 MHz (ver 6.4).

- El contador va de 0 a período − 1 (compara por igualdad con period_m1); la salida vale 1 mientras contador < umbral.
- El período se elige según freq_sel y el umbral se calcula sin divisor: `duty · (período / 100)`, con una multiplicación por constante (1250, 125 o (duty·25)>>1).
- El cálculo está registrado y `cfg_valid` se retrasa un ciclo (`cfg_valid_d`) para cargar los valores ya calculados y reiniciar el contador.
- Con duty 0 % la salida queda siempre en 0; con 100 %, siempre en 1.
- La salida está registrada (sale de un flip-flop) para evitar glitches en un pin externo.
- Los registros son de 20 bits porque el período más largo (125 000 ciclos) cabe en 20 bits.
- Expone dbg_period, dbg_thresh y dbg_cnt para el ILA.

4.5 pwm_top y pwm_top_vio

Conectan los módulos, ponen el buffer de reloj global (BUFG) y sincronizan el reset con 2 flip-flops. Los LEDs 0 y 1 muestran freq_sel y el LED 2 replica la salida PWM.

pwm_top_vio es la versión usada para la prueba en placa: agrega un VIO cuya salida de 1 bit se combina (OR) con el botón BTN0 para formar el reset, y cuya entrada de 3 bits monitorea `led[2:0]`. Las señales de debug llevan el atributo `mark_debug` para insertar el ILA.

5. Simulación

Metodología

Todos los componentes del bloque principal y el bloque principal completo se simularon con ModelSim, usando un testbench por cada unidad bajo prueba. Los testbenches son autoverificables: generan el reloj de 125 MHz (período de 8 ns), aplican los estímulos, comparan los resultados con los valores esperados y cuentan los errores. Al terminar imprimen en la consola TB <nombre>: PASSED si

no hubo errores, o FAILED con el detalle de cada diferencia. Un temporizador de guarda evita que la simulación quede colgada.

Para los módulos UART se usa 115200 baudios, es decir 1085 ciclos de reloj por bit (8,68 µs) y 86,8 µs por trama de 10 bits. Los scripts compile.do y run_<testbench>.do compilan el diseño y abren la ventana de ondas con las señales ya agrupadas.

Ventanas utilizadas. La ventana **Objects** lista las señales de la unidad seleccionada con su valor; se usa para mostrar el estado al final de la simulación (contadores, configuración y banderas). La ventana **Wave-Default** muestra la evolución temporal de las señales; en cada caso se indican los instantes de interés para hacer zoom.

Resumen de las simulaciones

Simulación	Testbench	Unidad bajo prueba	Tiempo simulado	Resultado esperado
Componente	`tb_uart_rx`	`uart_rx`	1,16 ms	PASSED
Componente	`tb_uart_tx`	`uart_tx`	0,72 ms	PASSED
Componente	`tb_uart_cmd_parser`	`uart_cmd_parser`	53,6 µs	PASSED
Componente	`tb_pwm_generator`	`pwm_generator`	9,38 ms	PASSED
Bloque principal	`tb_pwm_top`	`pwm_top`	10,15 ms	PASSED

Simulación de los componentes

uart_rx

Objetivo. Verificar que el receptor reconstruye bytes 8N1 a 115200 baudios, entrega un pulso de un ciclo en data_rdy con el byte correcto y descarta un falso start.

Estímulos

- Reset de 80 ns.
- Diez bytes con patrones distintos: `0x00`, `0xFF`, `0x55`, `0xAA`, `0x44 ('D')`, `0x37 ('7')`, `0x0A (LF)`, `0x0D (CR)`, `0x80`, `0x01`.
- Dos bytes consecutivos sin espacio entre tramas: `0xA5`, `0x5A`.
- Falso start: rx en 0 durante 1,085 µs (un octavo de bit), mucho menos que medio bit.
- Un byte válido después del falso start: `0x31 ('1')`.

Verificaciones automáticas

- Cada trama genera exactamente un pulso de data_rdy y data_out coincide con el byte enviado.
- Los bytes consecutivos generan dos pulsos.
- El falso start no genera data_rdy y el receptor sigue funcionando después.

Instantes de interés

Instante (µs)	Evento	Señales a observar
0,16 – 86,99	Primer byte, 0x00: trama completa con start, 8 datos y stop.	rx, state, clk_cnt, bit_idx, data_rdy (pulso en ≈ 82,6 µs), data_out
86,99 – 173,82	Byte 0xFF: todos los bits de datos en 1.	rx, data_out = FF
347,49 – 434,32	Byte 0x44 ('D'), carácter del protocolo.	rx, data_out = 44
868,48 – 1042,08	Bytes consecutivos 0xA5 y 0x5A.	data_rdy (pulsos en ≈ 950,9 µs y ≈ 1037,7 µs), data_out
1042,11 – 1043,20	Falso start de 1,085 µs.	rx, state (START y vuelve a IDLE), data_rdy sin pulso
1069,24 – 1156,04	Byte 0x31 después del falso start.	data_out = 31, data_rdy

Valores esperados en Objects al final de la simulación

`rx_cnt = 13` (10 bytes + 2 consecutivos + 1 posterior al falso start), `last_byte = 31` (hexadecimal), `data_out = 31`, `data_rdy = 0` y `done = TRUE`.

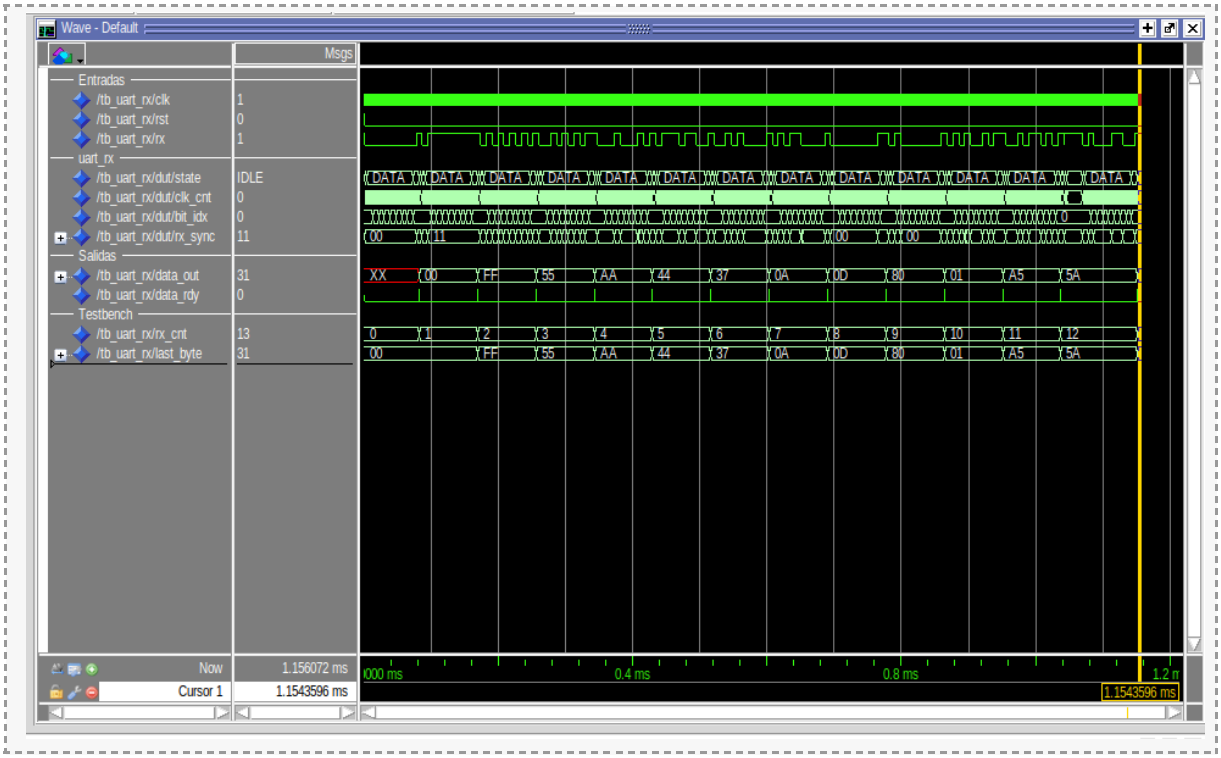


Figura 4. Simulación de uart_rx con los valores finales (rx_cnt = 13, last_byte = 31).

Cuando rx baja (bit de start) la máquina de estados pasa de IDLE a START, espera medio bit y confirma que la línea sigue en cero. En DATA muestrea cada 1085 ciclos (clk_cnt de 0 a 1084) incrementando bit_idx de 0 a 7. En STOP emite un pulso de un ciclo en data_rdy con el byte en data_out.

El pulso en cero de 1,085 µs lleva al receptor a START; al cumplirse medio bit la línea ya volvió a 1, por lo que vuelve a IDLE sin generar data_rdy. A continuación el byte 0x31 se recibe con normalidad.

uart_tx

Objetivo. Verificar que el transmisor serializa un byte en una trama 8N1 al recibir un pulso en send, que busy refleja la transmisión y que un send con el transmisor ocupado se ignora.

Estímulos

- Reset de 80 ns.
- Siete bytes enviados con un pulso de send de un ciclo: `0x4F ('O'), 0x4B ('K'), 0x0D, 0x0A, 0x00, 0xFF, 0xA5`.
- Un segundo pulso de send con `data_in = 0x11` durante la transmisión de otro byte (`0x4F`).

Verificaciones automáticas

- En reposo, tx = 1 y busy = 0.

- Un monitor decodifica la línea tx muestreando al centro de cada bit: start en 0, ocho bits de datos con el menos significativo primero, stop en 1; el byte decodificado coincide con el pedido.
- busy vale 1 durante la trama y vuelve a 0 al terminar, con tx en 1.
- El send con busy = 1 no genera otra trama ni modifica el byte en curso.

Instantes de interés

Instante (µs)	Evento	Señales a observar
0,11 – 86,95	Primera trama: 0x4F ('O').	send, busy, tx, state, clk_cnt, bit_idx
86,95 – 173,78	Segunda trama: 0x4B ('K').	tx, busy
173,78 – 347,44	Tramas 0x0D (CR) y 0x0A (LF).	tx, busy
347,44 – 607,94	Tramas 0x00, 0xFF y 0xA5 (casos extremos de datos).	tx
607,94 – 694,8	Trama 0x4F con un send adicional en ≈ 634 µs.	send (segundo pulso), busy = 1, tx sigue enviando 0x4F
≈ 720,8	Fin de la simulación.	tx = 1, busy = 0

Valores esperados en Objects al final de la simulación

`frm_cnt = 8` (7 tramas + la del send ignorado), `frm_byte = 4F`, `start_ok = TRUE`, `stop_ok = TRUE`, `tx = 1`, `busy = 0` y `done = TRUE`.

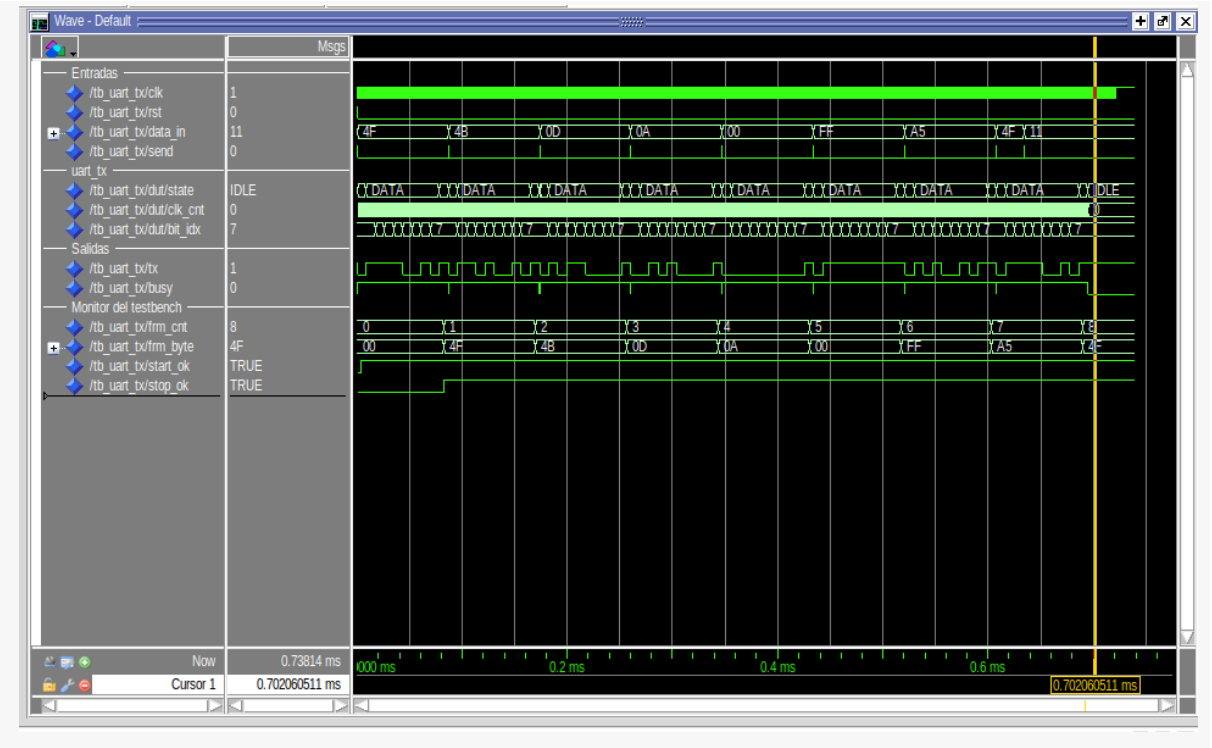


Figura 5. Simulación de uart_tx: ventana Objects con los valores finales (frm_cnt = 8, frm_byte = 4F).

Un pulso de un ciclo en send lleva a la máquina de estados de IDLE a START; busy sube y tx baja para el bit de start. Luego DATA envía los ocho bits, con el menos significativo primero, durante 1085 ciclos cada uno; STOP deja tx en 1 y busy vuelve a 0 al terminar.

El segundo pulso de send llega con la trama de 0x4F en curso (busy = 1). Como la máquina de estados solo atiende send en IDLE, el pulso se ignora: la trama continúa con 0x4F y no aparece ninguna trama adicional.

uart_cmd_parser

Objetivo. Verificar la interpretación de los comandos F1 a F3 y Dxx, la actualización de freq_sel y duty_cycle, el pulso cfg_valid, las respuestas OK y ERR y que un comando inválido no altere la configuración.

Estímulos

- El testbench reemplaza a uart_rx entregando cada carácter con un pulso de rx_rdy de un ciclo, y modela a uart_tx con una señal tx_busy de 20 ciclos por byte transmitido.
- Comandos válidos: `F2`, `F3`, `F1`, `F2` (terminado en CR), `D75`, `D0`, `D100`, `D5` (terminado en CR), `D007` y `D60` (este último después de los errores).
- Comandos inválidos: `D150` (fuera de rango), `F4`, `X`, `D` sin dígitos y `Da`.
- Una línea vacía (solo LF).

Verificaciones automáticas

- Valores por defecto después del reset: freq_sel = "01" y duty_cycle = 50.
- Cada comando válido produce un único ciclo de cfg_valid, actualiza la configuración y responde OK seguido de CR y LF (4 bytes).
- Cada comando inválido responde ERR seguido de CR y LF (5 bytes), sin cfg_valid y sin cambiar la configuración.
- Después de los errores el parser sigue aceptando comandos, y la línea vacía no genera respuesta.

Instantes de interés

Instante (µs)	Evento	Señales a observar
0,16 – 13,51	Comandos F2, F3, F1 y F2 (con CR).	rx_data, parse_state, freq_sel (01 → 10 → 11 → 01 → 10), cfg_valid, tx_data (O, K, CR, LF)
13,51 – 26,99	Comandos D75, D0, D100 y D5 (con CR).	duty_cycle (75, 0, 100, 5), num_acc, cfg_valid
26,99 – 30,43	Comando D007 (tres dígitos).	duty_cycle = 7
30,43 – 46,96	Comandos inválidos: D150, F4, X, D y Da.	parse_state pasa por ST_ERROR, tx_data (E, R, R, CR, LF), cfg_valid sin pulsos, duty_cycle = 7 y freq_sel = 10 sin cambios
46,96 – 50,35	Comando D60, válido después de los errores.	duty_cycle = 60, cfg_valid, respuesta OK
50,35 – 53,59	Línea vacía (LF).	sin respuesta ni cfg_valid

Valores esperados en Objects al final de la simulación

`tx_cnt = 65` (9 respuestas OK de 4 bytes + 5 ERR de 5 bytes + el OK de D60), `cfg_cnt = 10` (10 comandos válidos), `freq_sel = 10`, `duty_cycle = 0111100` (60) y `done = TRUE`.

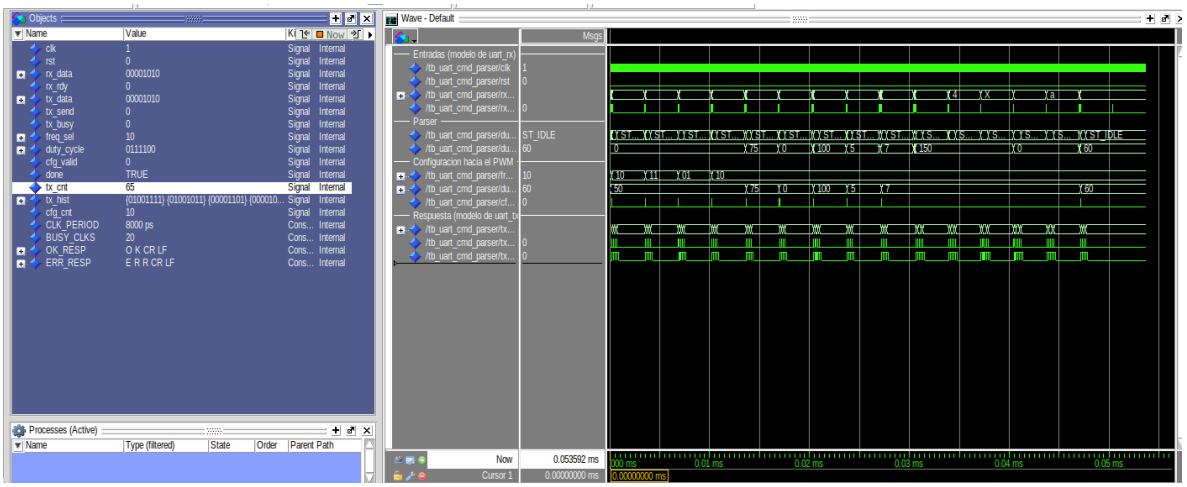


Figura 6. Simulación de uart_cmd_parser: ventana Objects con los valores finales (tx_cnt = 65, cfg_cnt = 10).

Con cada comando válido el parser cambia freq_sel o duty_cycle y activa cfg_valid durante un solo ciclo. Después carga la respuesta y la envía byte a byte por tx_data / tx_send, esperando que tx_busy valga 0 entre bytes: O, K, CR y LF. En D75 se ve cómo num_acc pasa de 7 a 75 antes de aplicar el valor.

Ante cada comando inválido el parser pasa por ST_ERROR y envía E, R, R, CR y LF. cfg_valid no se activa y freq_sel y duty_cycle conservan sus valores. El comando D60 posterior se acepta con normalidad, y la línea vacía no produce respuesta.

pwm_generator

Objetivo. Verificar el cálculo de período y umbral para las tres frecuencias y todos los valores de duty cycle, la forma de onda de la salida, la frecuencia generada y el reinicio del contador al cargar una nueva configuración.

Estímulos

- Reset de 5 ciclos.
- Las 303 combinaciones de frecuencia (F1 = 1 kHz, F2 = 10 kHz y F3 = 100 kHz) y duty cycle de 0 a 100, cargadas como lo hace el parser: freq_sel, duty_cycle y cfg_valid cambian juntos y cfg_valid dura un ciclo.
- freq_sel = "00" (valor no usado).
- Mediciones de la forma de onda con duty 0, 25, 50, 75 y 100 % en F2 y F3, y con 0, 50 y 100 % en F1.
- Medición del período entre dos flancos ascendentes con duty 50 % para las tres frecuencias.
- Cambio de configuración con el contador avanzado.

Verificaciones automáticas

- Después del reset: período = 125000, umbral = 62500 (1 kHz y 50 %).

- En las 303 combinaciones, dbg_period es el período de la frecuencia y dbg_thresh vale el período por duty dividido 100 (parte entera).
- freq_sel = "00" usa el período de 1 kHz.
- La cantidad de ciclos de pwm_out en alto durante un período es igual al umbral, incluidos los casos de 0 % (siempre 0) y 100 % (siempre 1).
- El período medido es de 125000, 12500 y 1250 ciclos para F1, F2 y F3.
- Al cargar una configuración el contador se reinicia.

Instantes de interés

Instante (µs)	Evento	Señales a observar
0,06 – 12,18	Carga de las 303 combinaciones, una cada 40 ns.	freq_sel, duty_cycle, cfg_valid, cfg_valid_d, dbg_period, dbg_thresh
12,18 – 12,22	freq_sel = 00 (valor no usado).	dbg_period = 125000, dbg_thresh = 50000
12,26 – 1012,38	F2 (10 kHz) con duty 0, 25, 50, 75 y 100 %, 200 µs cada uno.	pwm_out, dbg_cnt, dbg_thresh
1012,38 – 1112,50	F3 (100 kHz) con duty 0, 25, 50, 75 y 100 %, 20 µs cada uno (25 %: 1032 – 1052 µs; 75 %: 1072 – 1092 µs).	pwm_out, dbg_cnt
1112,50 – 7112,53	F1 (1 kHz) con duty 0, 50 y 100 %, 2 ms cada uno.	pwm_out
7112,53 – 9332,60	Medición del período entre flancos ascendentes para F1, F2 y F3 (duty 50 %).	pwm_out, dbg_period
9332,60 – 9382,68	Reinicio: en F1 el contador avanza durante 50 µs (hasta ≈ 6250) y al cargar F3 vuelve a 0.	dbg_cnt, cfg_valid, cfg_valid_d

Valores esperados en Objects al final de la simulación

`freq_sel = 11`, `duty_cycle = 0110010` (50), `dbg_period = 1250`, `dbg_thresh = 625`, `dbg_cnt` con un valor bajo (el contador acaba de reiniciarse), `cfg_valid = 0` y `done = TRUE`.

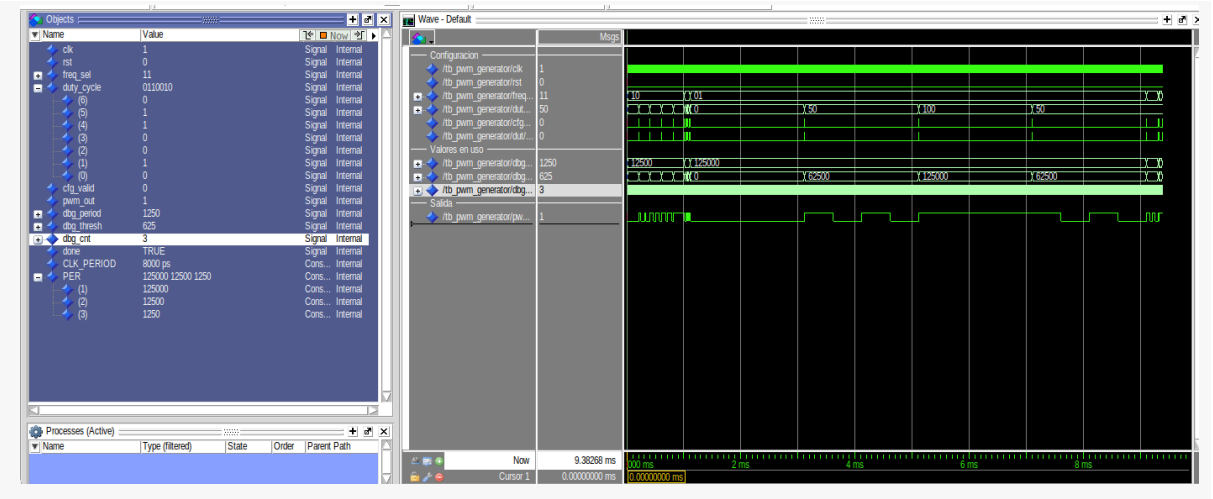


Figura 7. Simulación de pwm_generator: ventana Objects con los valores finales (período 1250 y umbral 625).

Con cada pulso de cfg_valid, un ciclo después cfg_valid_d carga los valores ya calculados: dbg_period toma 125000, 12500 o 1250 según freq_sel y dbg_thresh recorre el período por duty dividido 100. El cálculo está registrado, por lo que agrega un ciclo de latencia.

A 100 kHz el período es de 1250 ciclos (10 µs). Con 25 % el umbral es 312 y pwm_out permanece en alto 312 ciclos por período; con 75 % el umbral es 937 y la salida permanece en alto 937 ciclos. Para ver dbg_cnt como una rampa, clic derecho sobre la señal y Format → Analog (automatic).

En F1 el contador avanza sin interrupciones durante 50 µs hasta cerca de 6250. Al cargar F3 con cfg_valid_d el contador vuelve a 0 y empieza a contar el nuevo período de 1250 ciclos.

Simulación del bloque principal: pwm_top

Objetivo. Verificar el sistema completo: comandos recibidos por uart_rx a 115200 baudios, respuestas transmitidas por uart_tx, indicación de freq_sel en los LEDs y generación de la señal PWM con la frecuencia y el duty cycle pedidos.

El testbench instancia pwm_top y reemplaza al BUFG por un modelo de simulación (unisim_stub.vhd). La entrada uart_rx se maneja con tramas 8N1 reales, un monitor decodifica los bytes que salen por uart_tx y la salida pwm_out se mide en ciclos de reloj.

Estímulos y verificaciones

- Reset por botón (rst_btn) al inicio y al final.
- Secuencia de comandos: `D75` (con la frecuencia inicial, 1 kHz), `F3` (100 kHz), `F2` (10 kHz), `D25`, `D150` (inválido), `X` (inválido), `D100` y `D0`.
- Cada comando válido responde OK y cada inválido responde ERR, decodificados desde uart_tx.
- led(1:0) refleja freq_sel: 01 después del reset, 11 con F3 y 10 con F2.

- Período y ciclos en alto de pwm_out: 125000 y 93750 (1 kHz, 75 %); 1250 y 937 (100 kHz, 75 %); 12500 y 9375 (10 kHz, 75 %); 12500 y 3125 (10 kHz, 25 %), que se mantienen después de D150 y de X.
- Con D100 la salida queda fija en 1 y con D0 fija en 0.
- Después del reset final los LEDs vuelven a 01 y la salida vuelve a oscilar (1 kHz, 50 %).

Instantes de interés

Instante (µs)	Evento	Señales a observar
0 – 20,2	Reset por botón y arranque.	rst_btn, led, parse_state
20,2 – 723,4	Comando D75 (entra por uart_rx hasta ≈ 367 µs; cfg_valid en ≈ 363 µs) y respuesta OK por uart_tx.	uart_rx, parse_state, cfg_valid, uart_tx, duty_cycle
1363,1 – 2363,1	Un período de 1 kHz con 75 % de duty: 750 µs en alto.	pwm_out, dbg_period = 125000, dbg_thresh = 93750
2363,1 – 2892,7	Comando F3 (100 kHz) y respuesta OK; led(1:0) pasa a 11.	uart_rx, freq_sel, led, uart_tx
2902,4 – 2912,4	Un período de 100 kHz con 75 %: 10 µs, 7,5 µs en alto.	pwm_out, dbg_period = 1250, dbg_thresh = 937
2912,4 – 3442,0	Comando F2 (10 kHz) y respuesta OK; led(1:0) pasa a 10.	freq_sel, led
3581,7 – 4424,6	Comando D25 y un período con 25 %: 100 µs, 25 µs en alto.	duty_cycle, pwm_out, dbg_thresh = 3125
4424,6 – 6124,6	Comandos inválidos D150 y X: respuesta ERR y la salida no cambia.	uart_tx (E, R, R, CR, LF), cfg_valid sin pulsos, pwm_out
6124,6 – 8131,0	D100 (salida fija en 1) y D0 (salida fija en 0).	pwm_out
8131,0 – 10151,2	Reset por botón: vuelve a F1 y 50 %.	led, pwm_out, dbg_period = 125000

Valores esperados en Objects al final de la simulación

`tx_cnt = 34` (6 respuestas OK de 4 bytes + 2 ERR de 5 bytes), `led(1:0) = 01` (con led(2) igual a pwm_out), `rst_btn = 0`, `uart_rx = 1` y done = TRUE.

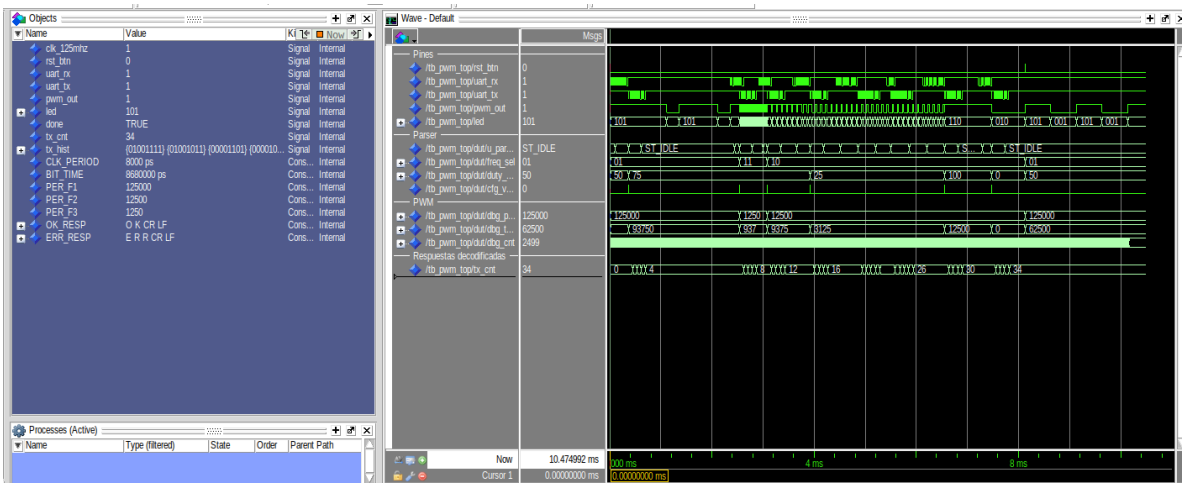


Figura 8. Simulación de pwm_top: ventana Objects con los valores finales (tx_cnt = 34, led(1:0) = 01).

Se distinguen los comandos que entran por uart_rx, las respuestas que salen por uart_tx, los cambios de led(1:0) con F3 y F2, y los cambios de la señal pwm_out: 1 kHz, luego 100 kHz, luego 10 kHz, las salidas fijas de D100 y D0, y el regreso al estado inicial tras el reset final.

Las cuatro tramas D, 7, 5 y LF entran por uart_rx. Al recibir el fin de línea el parser valida el valor (75 ≤ 100), activa cfg_valid durante un ciclo y actualiza duty_cycle. A continuación se transmiten por uart_tx los bytes O, K, CR y LF.

Después de F3, pwm_out tiene un período de 1250 ciclos (10 µs) y permanece en alto 937 ciclos (≈ 7,5 µs), es decir el 75 % configurado antes con D75. dbg_period y dbg_thresh muestran 1250 y 937.

Conclusiones de la simulación

- Los cuatro componentes (uart_rx, uart_tx, uart_cmd_parser y pwm_generator) se simularon por separado con testbenches autoverificables y cumplen las verificaciones planteadas.
- El bloque principal pwm_top se simuló con tramas UART reales a 115200 baudios: los comandos válidos configuran la frecuencia y el duty cycle, los inválidos devuelven ERR sin alterar la configuración y la señal PWM medida coincide con los períodos y umbrales esperados.
- Con estos resultados se pasó a la etapa de síntesis, implementación y prueba en la placa.

6. Síntesis e implementación

6.1 Flujo en Vivado 2018.3

El diseño se sintetizó e implementó en Vivado 2018.3 para la Arty Z7-10 y se generó el archivo de configuración (.bit). El reloj se restringe con un período de 8 ns (125 MHz) en el pin H16.

6.2 Asignación de pines

Señal	Pin	Función
`clk_125mhz`	H16	Reloj de 125 MHz
`rst_btn`	D19	BTN0, reset activo en alto
`uart_rx`	Y18	Pmod JA, pin 1: entrada desde el adaptador USB-TTL
`uart_tx`	Y19	Pmod JA, pin 2: salida hacia el adaptador USB-TTL
`pwm_out`	W14	Salida PWM (conector Pmod)
`led[0]`, `led[1]`, `led[2]`	R14, P14, N16	LD0, LD1 y LD2

Todos los pines usan LVCMOS33.

6.3 Uso de recursos de la FPGA

Tabla obtenida de Report Utilization. Incluye el ILA y el VIO usados para depuración.

Summary

Resource	Utilization	Available	Utilization %
LUT	1787	17600	10.15
LUTRAM	154	6000	2.57
FF	2679	35200	7.61
BRAM	56	60	93.33
DSP	1	80	1.25
IO	8	100	8.00

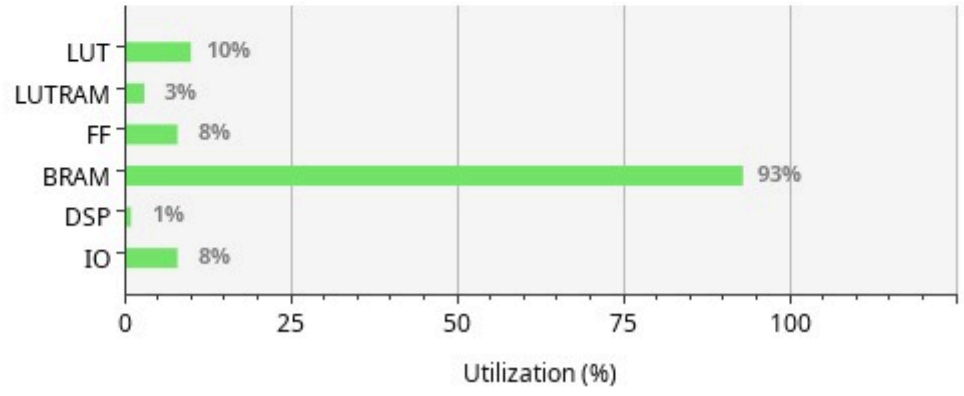


Figura 9. Uso de recursos de la FPGA.

6.4 Cierre de timing

Después de la síntesis, el reporte de timing mostró una violación de setup a 125 MHz (período de 8 ns). Se corrigió modificando pwm_generator.vhd; esta sección documenta el cambio.

Parámetro	Antes del cambio	Después del cambio
Setup WNS	−2,749 ns	**1.113ns**
Setup TNS	−40,837 ns	**0.000 ns**

Causa. El umbral se calculaba como `período . duty / 100` de forma combinacional (multiplicador de 20×7 bits y división por 100 sobre 27 bits) dentro de un único ciclo. Además, el contador comparaba con `period − 1` (resta y comparación) en cada ciclo.

Corrección. Se modificó solo pwm_generator.vhd, sin cambiar sus puertos, por lo que el resto del diseño no se tocó:

- Se eliminó el divisor: como los períodos son múltiplos de 100 (salvo 1250), el umbral es duty · (período/100), una multiplicación por constante. El resultado es idéntico a la división entera.
- El período, el período − 1 y el umbral se calculan en registros, y cfg_valid se retrasa un ciclo (cfg_valid_d) para cargar los valores ya calculados.
- El contador compara por igualdad con un registro period_m1 en lugar de restar y comparar.
- La carga de configuración reinicia el contador mediante if / else, como indicaba el comentario original.
- Se corrigieron las frecuencias para que coincidan con el protocolo: 1 kHz, 10 kHz y 100 kHz.

Efecto funcional. Un ciclo más de latencia (8 ns) entre la aceptación del comando y el cambio del PWM.

Verificación. Ver la sección 5: el testbench tb_pwm_generator.vhd comprueba el período y el umbral para todas las combinaciones de frecuencia y duty.

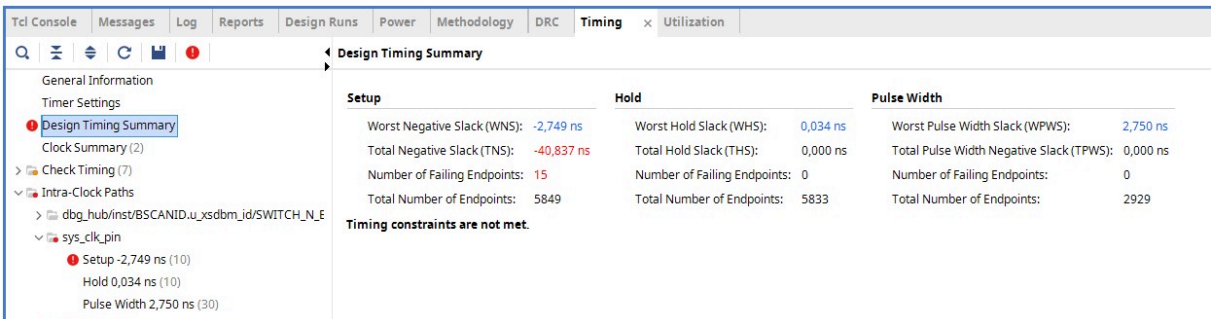


Figura 10. Primer reporte de tiempo

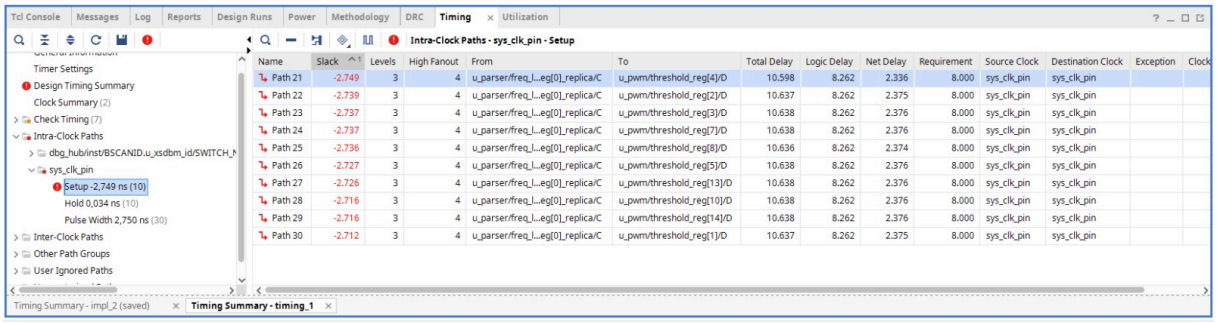


Figura 11. Detalle de reporte de tiempo

Luego de modificar el pwm_generator.vhd

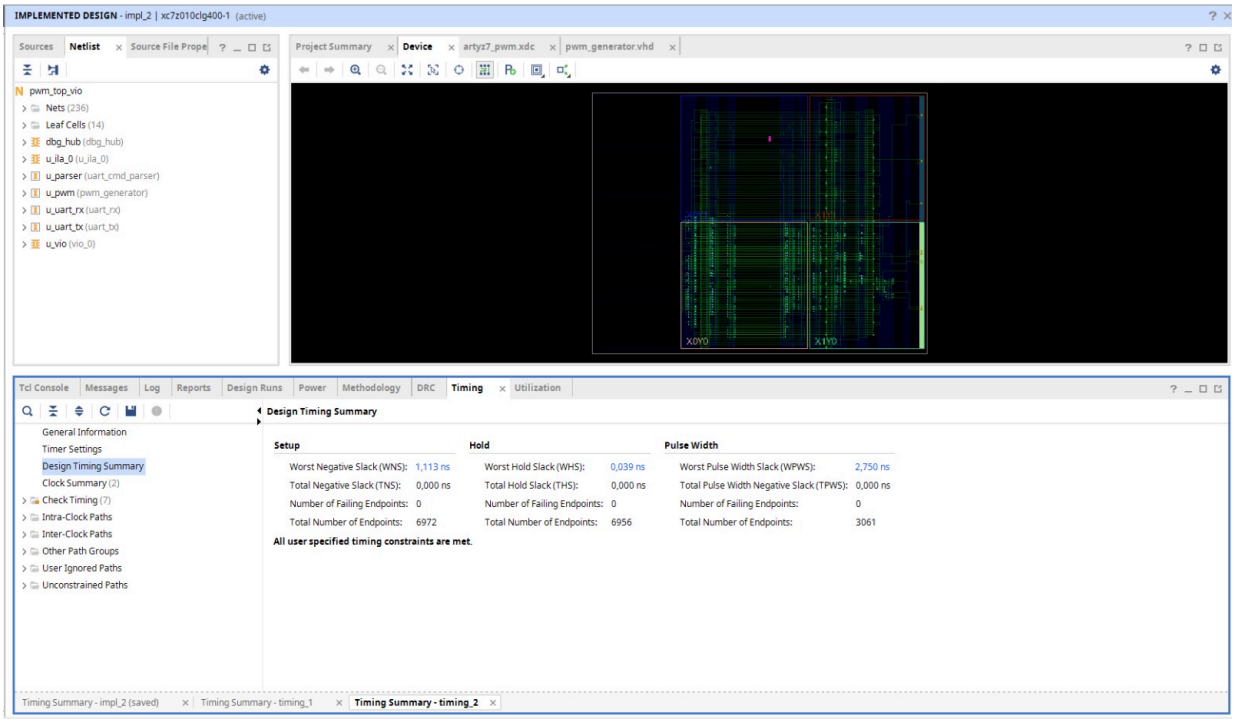


Figura 12. Reporte de tiempo luego de mejoras

6.5 Uso de BRAM del ILA y profundidad de captura

El ILA guarda las muestras en bloques de memoria BRAM de la FPGA. El diseño no declara memorias, por lo que el BRAM del reporte de recursos corresponde al ILA.

Cada muestra tiene 61 bits (dbg_thresh 20, dbg_cnt 20, dbg_period 20 y pwm_sig 1). La memoria necesaria es aproximadamente `profundidad × 61 bits`, por lo que el BRAM crece en proporción a la profundidad.

Profundidad	Ventana de captura (× 8 ns)	BRAM utilizado
16 384	≈ 131 µs	47 % (≈ 28 de 60 bloques)
32 768	≈ 262 µs	93 % (≈ 56 de 60 bloques)

Los porcentajes son los medidos en Vivado; la cantidad de bloques es una estimación a partir de los 60 BRAM36 de la FPGA.

En la siguiente imagen se ve la tabla de resumen del reporte de tiempo de la implementación en Vivado, con un una profundidad de

set_property C_DATA_DEPTH 32768 [get_debug_cores u_ila_0]

Luego de modificar la profundidad a

set_property C_DATA_DEPTH 16384 [get_debug_cores u_ila_0]

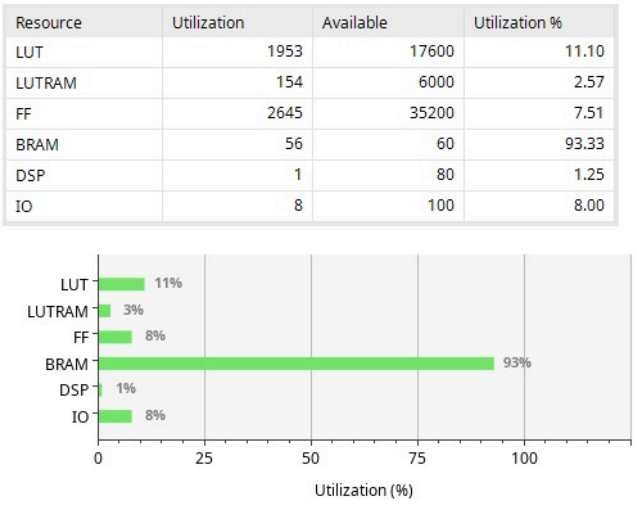


Figura 13. Uso de recursos con data depth de 32768

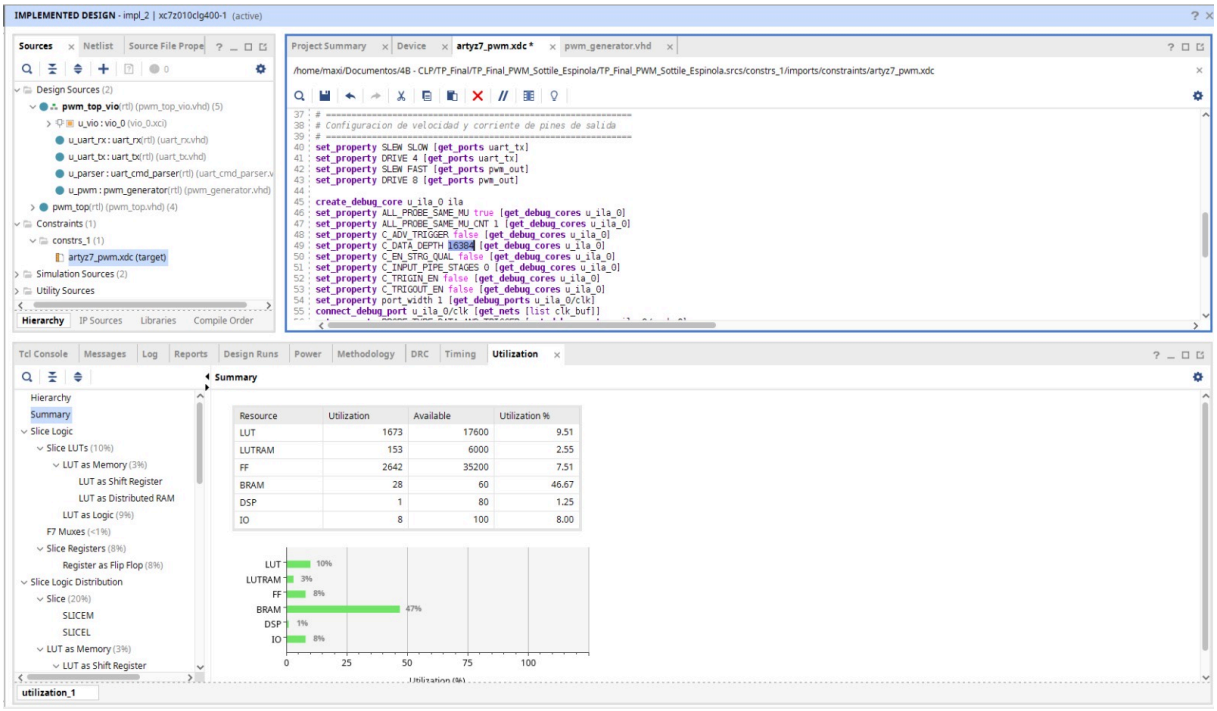


Figura 14. Mejora en el uso de recursos de BRAM con data depth de 16384

Períodos completos que entran en la ventana

Frecuencia	Período	Con 16 384	Con 32 768
1 kHz	1000 µs	0,13	0,26
10 kHz	100 µs	1,3	2,6
100 kHz	10 µs	13	26

La señal de menor frecuencia no se visualizaba bien con 16 384 muestras y mejoró al duplicar la profundidad a 32 768, pero con ese valor el BRAM llega al 93 %. Con el generador corregido, la frecuencia más baja es 1 kHz y ni con 32 768 muestras entra un período completo.

Por qué no se aumenta más la profundidad. Con 93 % casi no queda BRAM libre, lo que reduce el margen y puede dificultar la implementación, y la siguiente profundidad (65 536) ya no entra en la FPGA con 61 bits por muestra.

Alternativas para ver mejor las frecuencias bajas sin gastar más BRAM:

- Reducir el ancho de muestra: dbg_period y dbg_thresh solo cambian al recibir un comando y ocupan 40 de los 61 bits; dejando contador y salida (21 bits), la misma profundidad usa aproximadamente un tercio del BRAM.
- Calificación de almacenamiento (capture control): el ILA guarda una muestra solo cuando cambia la salida, y con el contador se obtienen período y duty con muy pocas muestras.
- Observar pwm_out con un osciloscopio para las frecuencias bajas y usar el ILA para el detalle interno (umbral, contador y cfg_valid).

7. Prueba en placa

Para verificar el funcionamiento en la placa se usó pwm_top_vio con un VIO y un ILA, y los comandos se enviaron desde una terminal serie por los pines JA1 y JA2. Esto se realizó gracias a la conexión remota con el servidor brindado por la cátedra.

1. Conexión de un adaptador USB-TTL de 3,3 V: su TX al pin JA1, su RX al pin JA2 y las masas unidas.
2. Terminal configurada a 115200 baudios, 8N1, enviando CR o LF al final de cada comando.
3. Programación de la FPGA desde el Hardware Manager con el .bit y el archivo de sondas (.ltx).
4. En el VIO: probe_in0 muestra led[2:0] y probe_out0 activa el reset virtual (1 = reset, 0 = suelto).
5. El ILA captura contador, umbral, período y salida PWM.

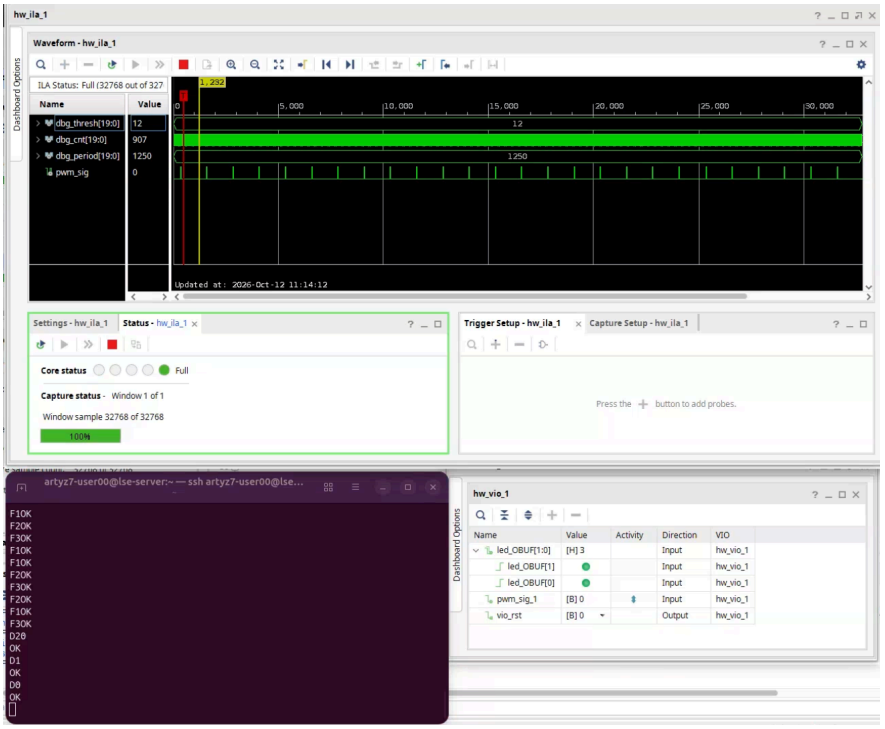


Figura 15. Prueba en placa con acceso remoto ILA y VIO

Nota: el VIO muestrea lento, por lo que led[2] se ve como un valor aislado y no como forma de onda; la forma de onda del PWM se observa con el ILA o con un osciloscopio.

8. Conclusiones y mejoras

Se implementó un generador PWM de frecuencia y duty cycle configurables por UART, con confirmación de cada comando, simulado, sintetizado y verificado en la placa con un VIO y un ILA. Durante la síntesis se detectó una violación de timing en el cálculo del umbral, que se corrigió registrando el cálculo y eliminando el divisor (sección 6.4).

Mejoras posibles:

- Aplicar la nueva configuración al final del período en curso, para evitar períodos acortados.
- Agregar una FIFO en la recepción y, opcionalmente, un bit de paridad.

Anexo. Archivos entregados

Archivo	Contenido
`uart_rx.vhd`	Receptor UART 8N1
`uart_tx.vhd`	Transmisor UART 8N1

Archivo	Contenido
`uart_cmd_parser.vhd`	Intérprete de comandos y generador de respuestas
`pwm_generator.vhd`	Generador PWM (versión corregida para cumplir timing)
`pwm_top.vhd`	Top level
`pwm_top_vio.vhd`	Top level con VIO para la prueba en placa
`artyz7_pwm.xdc`	Restricciones de pines, reloj y ILA
`tb_pwm_generator.vhd`	Testbench autoverificable del generador PWM